

Lecture 3 (16.10.2025)

← finishes lecture 2
→ starts lecture 3

WC Big-O notation (admission) → the max time an algorithm might take.
→ the upper bound

BC Big-Ω notation (not very useful) → the min time an algorithm might take.
→ the lower bound

AC Big-Θ most precise → the exact growth rate

• we don't look at small sample sizes → the behaviour is weird.

Bubblesort

WC: Big-O : upper bound: $O(n^2)$ list sorted in reverse order

BC: Big-Ω : lower bound $\Omega(n)$ list already sorted

AC: Big-Θ : tight bound $\Theta(n^2)$

$T(n) \in \Theta(f(n))$ if $\lim_n \frac{T(n)}{f(n)} = ct \neq 0$

$$\left. \begin{array}{l} T(n) = 13n^3 + 42n^2 + 2n \cdot \log_2 n + 3\sqrt{n} \\ f(n) = n^3 \end{array} \right\} \Rightarrow \lim_n \frac{T(n)}{f(n)} = 13. = ct$$

$\Rightarrow T(n) \in \Theta(n^3)$

or

$$\left. \begin{array}{l} T(n) \in O(n^3) \\ T(n) \in \Omega(n^3) \end{array} \right\} \Rightarrow T(n) \in \Theta(n^3)$$

$$\text{Run time} = \Theta(f(n)) \Leftrightarrow \begin{cases} \text{WC} = O(f(n)) \\ \text{BC} = \Omega(f(n)) \end{cases}$$

Summations

loops $T(n) = \sum_{i=1}^n f(i)$

nested loops $\Rightarrow$ nested sums

BC = WC = AC when it does not matter if the elem. are arranged.

AC: $T(n) = \frac{1+2+\dots+n}{n} = \frac{n+1}{2}$

sum of all work values number of scenarios

if the nr. could be in the first/second ... position

for i in range $(1, 2 * m - 1) // 1 \rightarrow 2 * m - 2$

$\log n \rightarrow$ binary search ($O(\log n) \approx \log_2 n$)

Exponential Search

$\hookrightarrow$ starts from the beginning
 $\hookrightarrow \log n$

Why would I use exp. search instead of bin. search if they have the same compl.?

If the number we search for is at the beginning of the list, there is no point in using binary search.

$$O(1) < O(\log n) < O(n) < O(\log^k n) < O(n^2) < O(n^k) < O(2^n)$$

$\downarrow$
constant
time

I 3ary tree

$$\begin{cases} N(h) = 1, h = 0 \\ N(h) = 1 + 3 * N(h-1), h \geq 1 \end{cases}$$

$$\Rightarrow N(h) = 1 + 3 + 3^2 + \dots + 3^h = \sum_{i=0}^h 3^i = \frac{3^{h+1} - 1}{3 - 1} \approx 3^h$$

$\downarrow$
" $\dots$ "

$\Rightarrow$ Each additional level increases the total amount of nodes exponentially.

II . Sum of numbers in a list.

$$T(n) = \begin{cases} 1, n = 0 \\ T(n-1) + 1, n > 0 \end{cases}$$

solve the
recurrence $\rightarrow$

$$T(n) = T(n-1) + 1 = n + 1. \in \Theta(n)$$

$$T(n-1) = T(n-2) + 1 = n - 1 + 1$$

$$T(n-2) = T(n-3) + 1 = n - 2 + 1$$

$\vdots$

$$T(2) = T(1) + 1 = 1 + 1 + 1 = 3$$

$$T(1) = T(0) + 1 = 1 + 1 = 2$$

III Tower of Hanoi $\rightarrow$ complexity depends on the number of disks



→ mathematically: $T(n) = \begin{cases} 1, & n=0 \\ 2T(n-1)+1, & n>1. \end{cases}$
 → there are zero disks (function calls once)
 → move the largest disk to dest. rod
 → move (n-1) disks to aux rod
 → move (n-1) disks from aux to dest.

BC: $T(1)=1$.

Solve the recurrence.

$$\begin{aligned} T(n) &= 2T(n-1)+1 = 2^{n+1}-1 \quad ? \Rightarrow T(n) = 2^n - 1 \in \Theta(2^n) \\ T(n-1) &= 2T(n-2)+1 = 2^{n-1+1}-1 \\ T(n-2) &= 2T(n-3)+1 = 2^{n-2+1}-1 \\ &\vdots \\ T(3) &= 2 \cdot T(2)+1 = 2 \cdot 3+1 = 7 = 2^3-1 \\ T(2) &= 2 \cdot T(1)+1 = 2 \cdot 1+1 = 3 = 2^2-1 \\ T(1) &= 2T(0)+1 = 2 \cdot 1+1 = 3 = 2^2-1 \\ T(0) &= 1 = 2^1-1 \end{aligned}$$

→ empirically: run the code.

Memory Space complexity

Iterative sum. $T(n)=1 \in \Theta(1)$

Recursive sum. $T(n) = \begin{cases} 0, & n=1 \\ T(n-1) + n-1, & n>1 \end{cases}$

Complexity also depends on CACHE. (but only practically; theoretically not included in calculations)

CPU → MEMORY → CACHE

↓
 fast! but small. → waits! (Bolt food)

ex: adding up the elements of a matrix.

→ theoretically, adding up the elem. by column should take the same time as doing it by row.

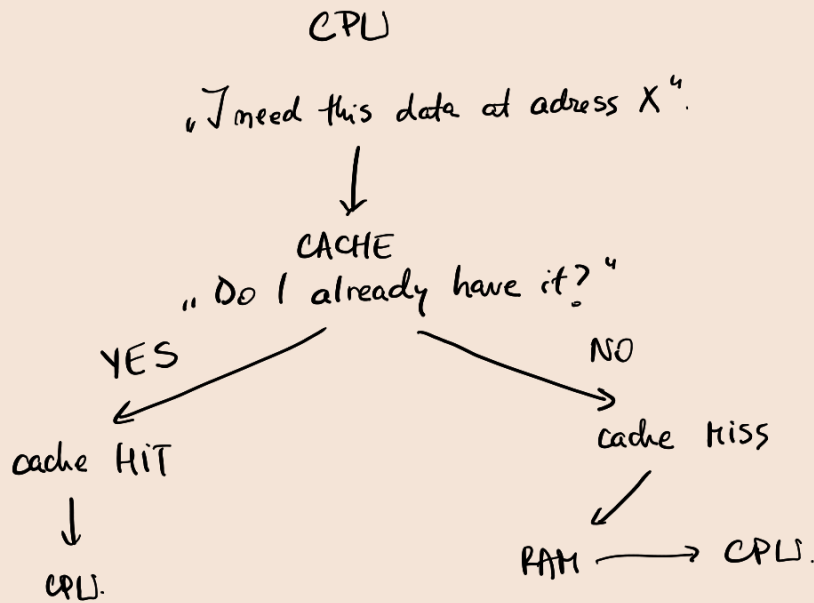
→ practically, column sum time $\neq$ row sum time

[a₀₀, a₀₁, a₀₂, a₁₀, a₁₁, a₁₂]
 cache hit cache miss cache hit

$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}$

miss
↓
go to RAM

The cache brings the whole block from RAM to itself
↳ has to go more often to RAM $\Rightarrow$ cache miss.



Searching. Sorting

Binary search vs. Exponential Search

Exponential Search

↳ starts from the beginning
↳ $\log n$

Why would I use exp. search instead of bin. search if they have the same compl.?

If the number we search for is at the beginning of the list, there is no point in using binary search.

! Iterative version faster than recursive

↓
function
↓
stack, memory